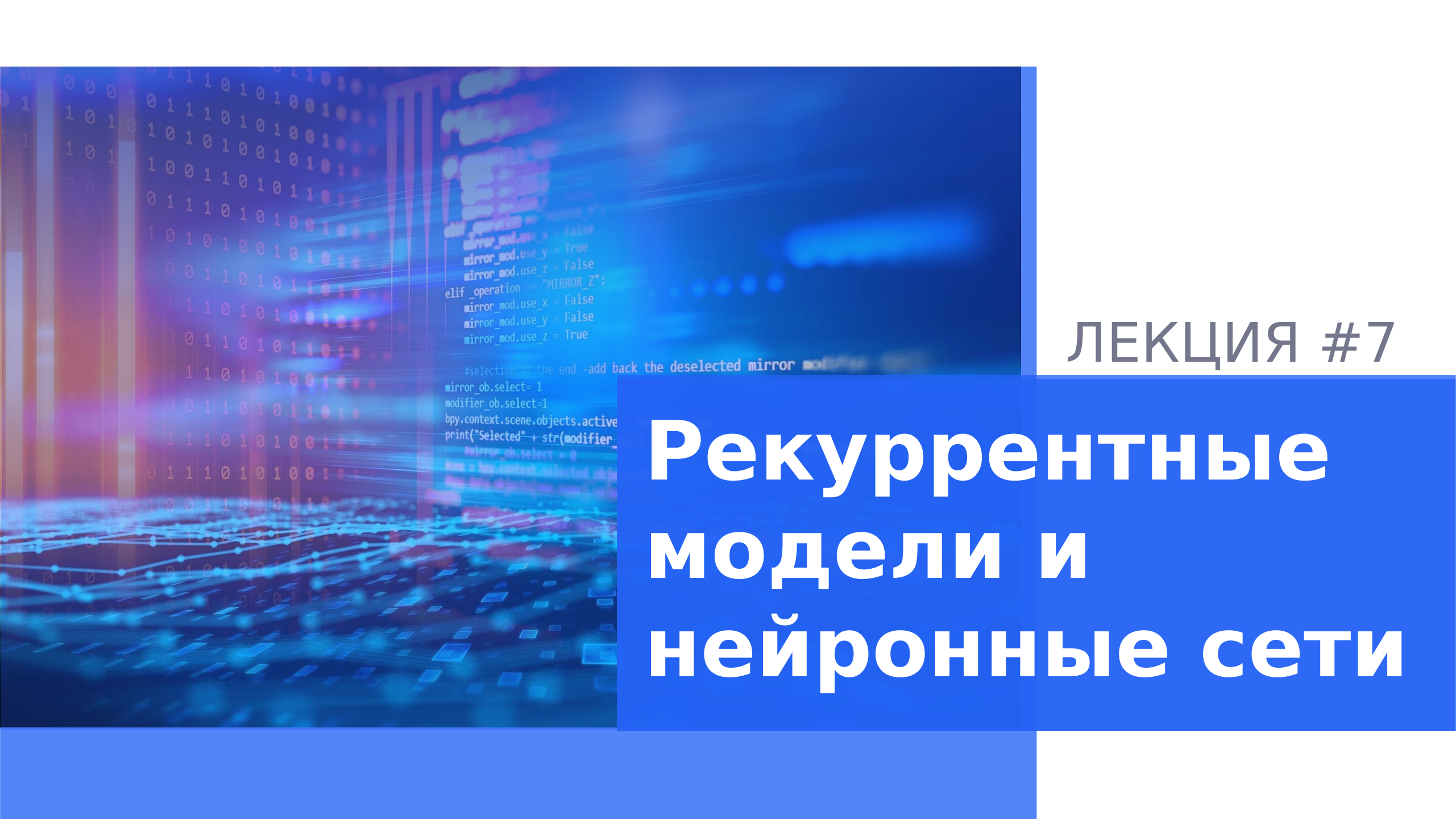


The background of the slide is a dark blue gradient. On the left side, there is a vertical strip of glowing binary code (0s and 1s) in a lighter blue color. In the center-left area, there is a semi-transparent rectangular box containing a Python code snippet. The code is as follows:

```
elif operation == "MIRROR_X":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = True  
    mirror_mod.use_z = False  
elif operation == "MIRROR_Z":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = False  
    mirror_mod.use_z = True  
  
#selection at the end -add back the deselected mirror modifier  
mirror_ob.select= 1  
modifier_ob.select=1  
bpy.context.scene.objects.active  
print("Selected" + str(modifier_...  
    mirror_ob.select = 0  
bpy.context.selected_obj  
bpy.data.objects[...]
```

On the right side, the text "ЛЕКЦИЯ #7" is written in a large, white, sans-serif font.

ЛЕКЦИЯ #7

Рекуррентные модели и нейронные сети

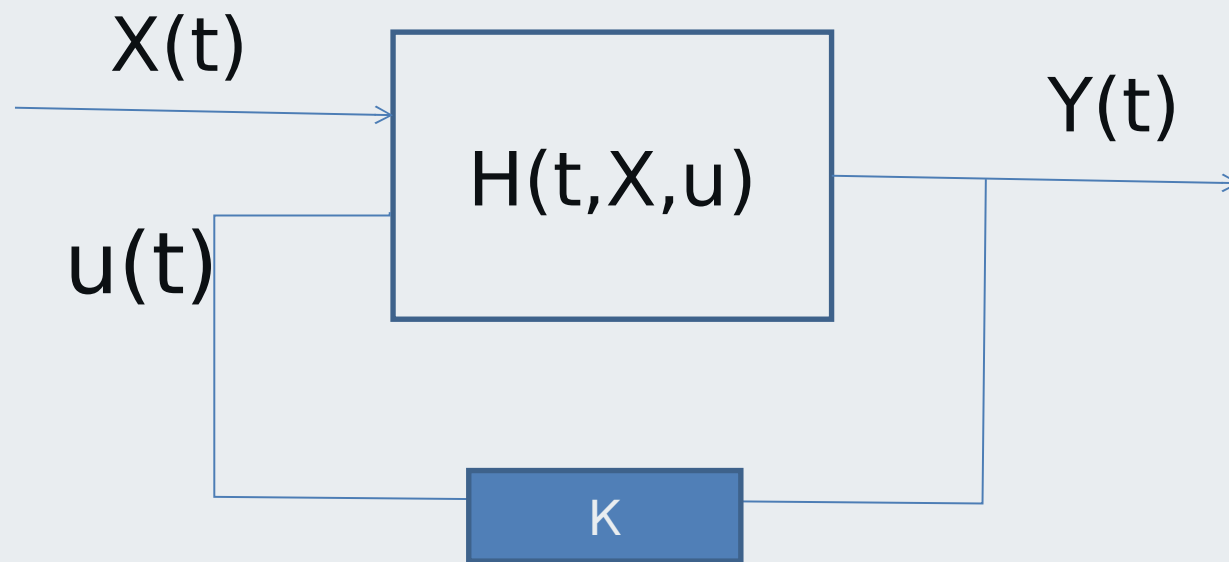
ПЛАН



- Модель рекуррентных связей в нейросети и влияние на результат работы
- Сеть Хопфилда - реализация и работа
- Обучение рекуррентных сетей: Backpropagation Through Time (BPTT).
- Сети долгой кратковременной памяти (Long short-term memory, LSTM).
- Рекуррентная сеть Gated Recurrent Unit (GRU).

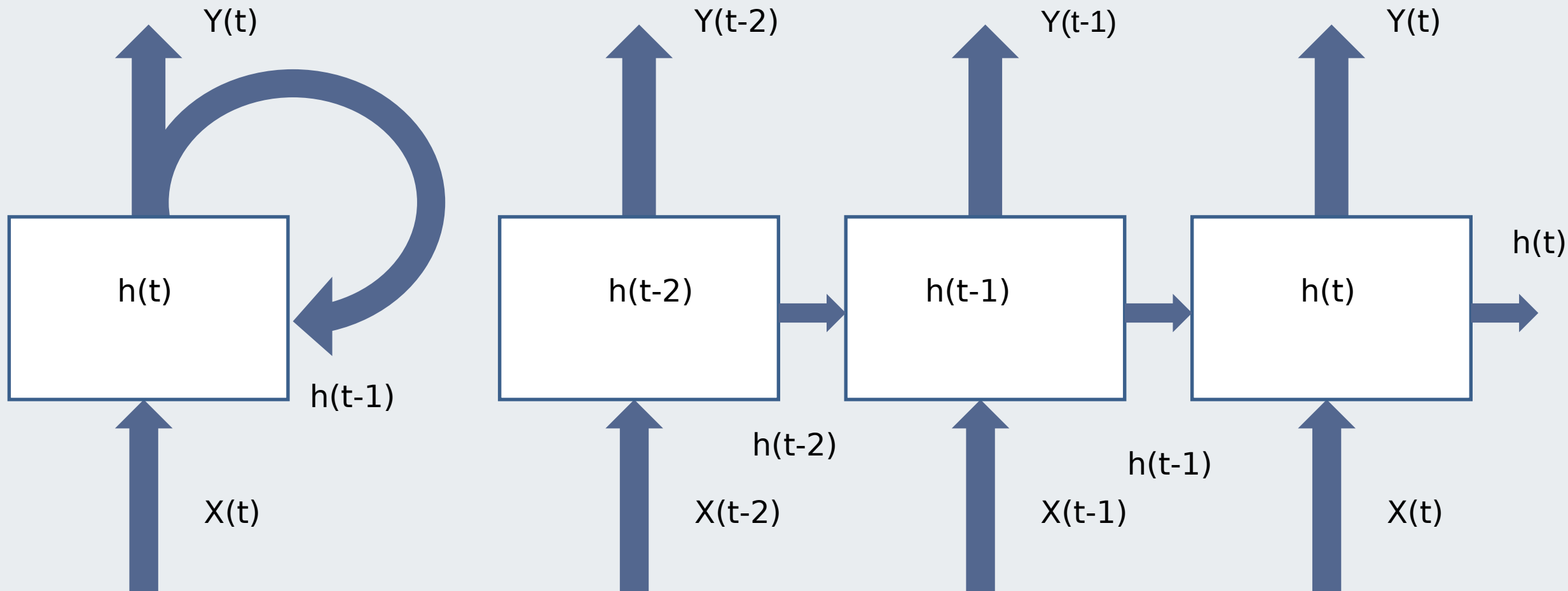


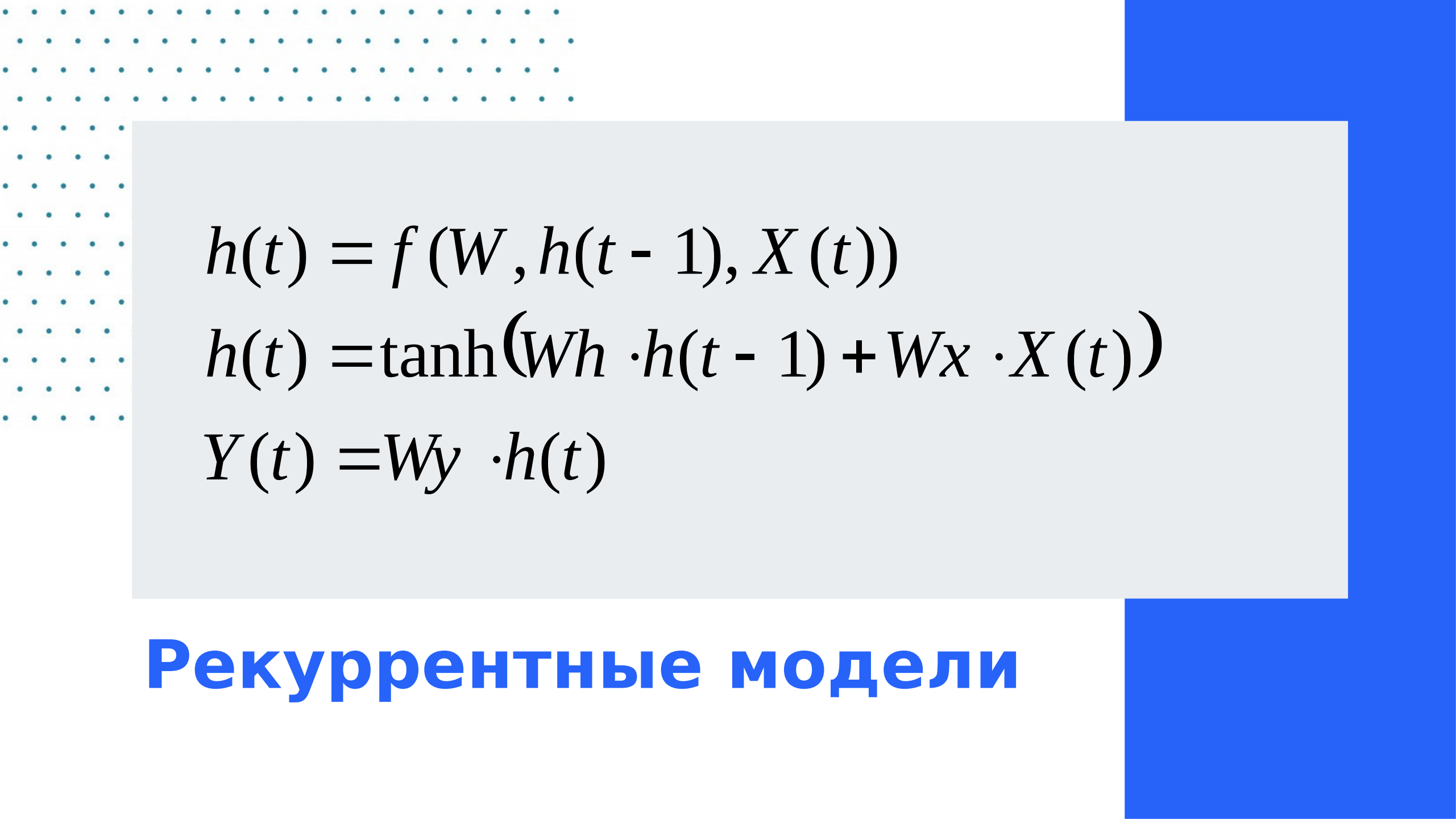
Рекуррентные модели



Динамическая система

Рекуррентные модели




$$h(t) = f(W, h(t-1), X(t))$$

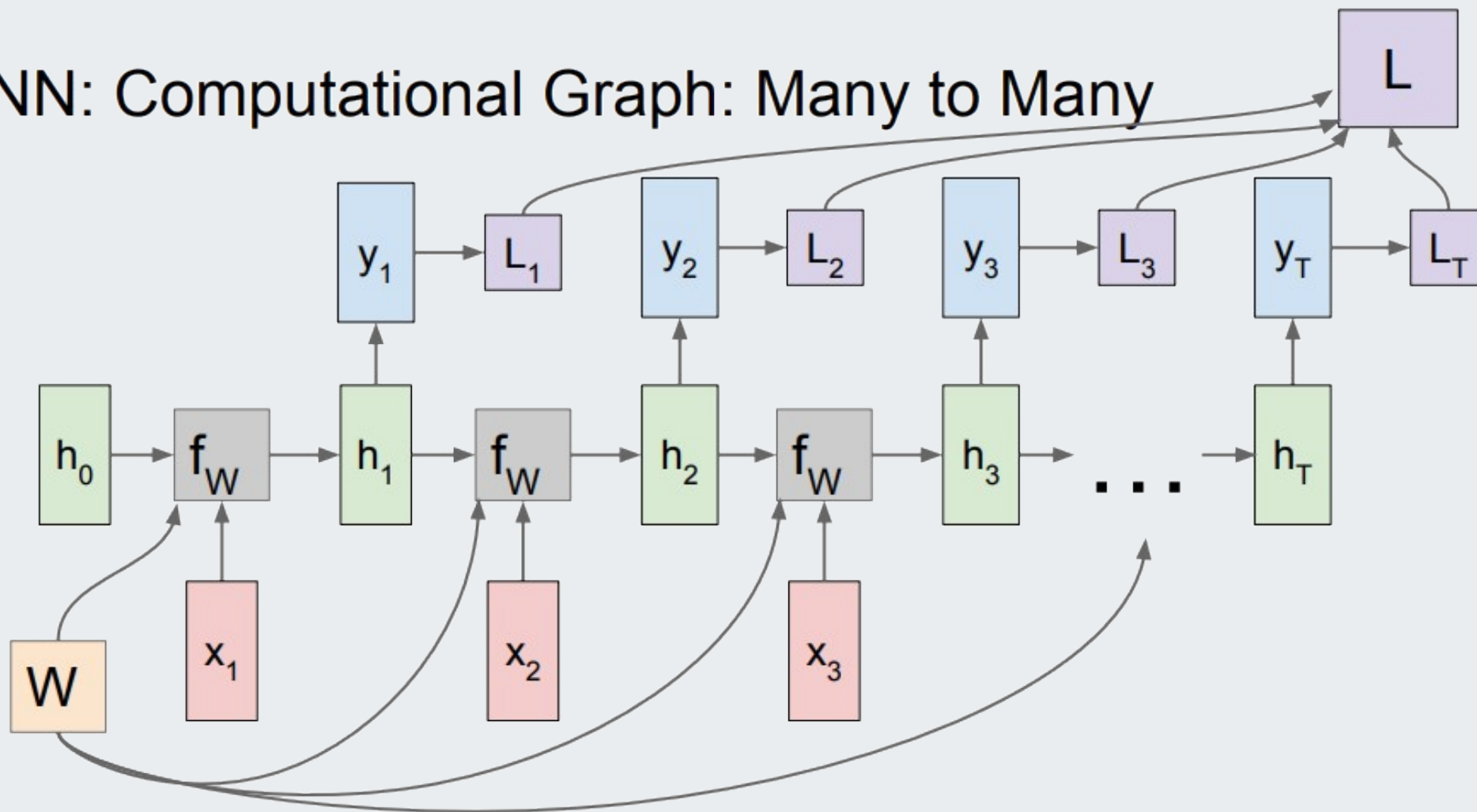
$$h(t) = \tanh(W_h \cdot h(t-1) + W_x \cdot X(t))$$

$$Y(t) = W_y \cdot h(t)$$

Рекуррентные модели

Рекуррентные модели

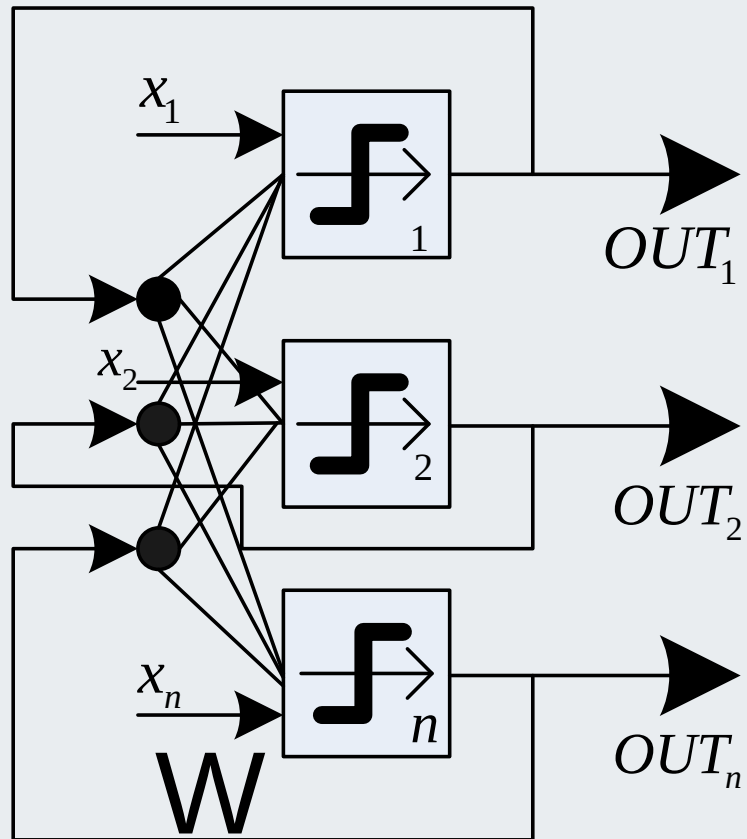
RNN: Computational Graph: Many to Many





Сеть Хопфилда

Сеть Хопфилда

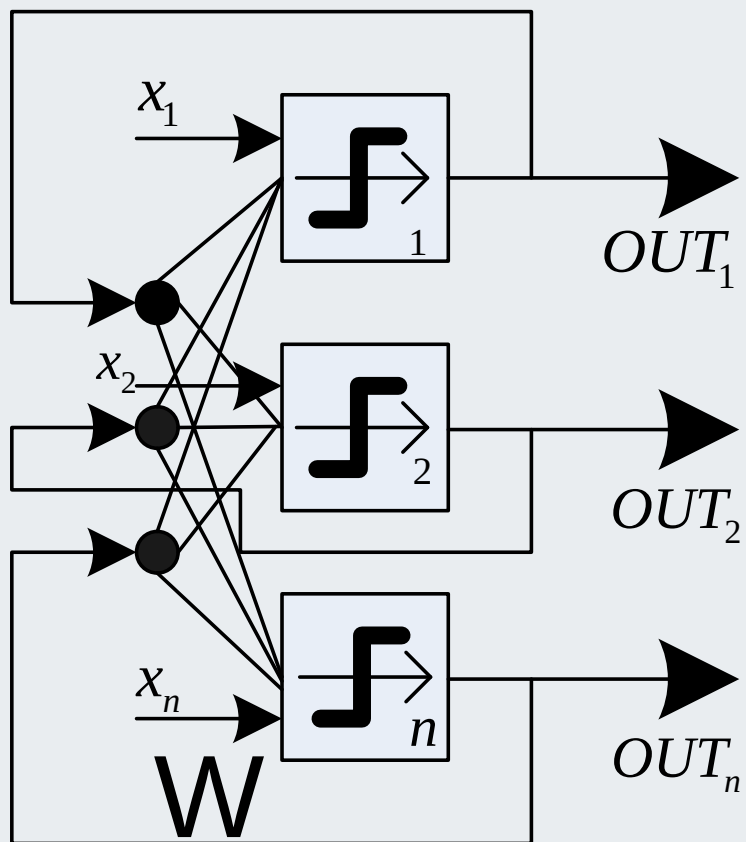


$$NET_j = \sum_{i \neq j} \omega_{ij} OUT_i + x_j$$

$$\begin{cases} OUT_j = 1, NET_j > T_j \\ OUT_j = 0(-1), NET_j < T_j \end{cases} \quad (T_j - \text{threshold})$$

$$E = -\frac{1}{2} \sum_i \sum_j \omega_{ij} OUT_i OUT_j - \sum_i x_i OUT_i + \sum_i T_i OUT_i$$

Сеть Хопфилда



$$NET_j = \sum \omega_{ij} OUT_i + x_j$$

$$\delta E_i = - \left[\sum_{j \neq i} \omega_{ij} OUT_j + x_i - T_i \right] \delta OUT_i$$

$$\delta E = - \left[NET_i - T_i \right] \delta OUT_i$$

$$\left[\begin{array}{l} NET_i > T_i \Rightarrow \delta OUT_i \geq 0 \Rightarrow \delta E_i \leq 0 \\ NET_i < T_i \Rightarrow \delta OUT_i \leq 0 \Rightarrow \delta E_i \leq 0 \\ NET_i = T_i \Rightarrow \delta E_i = 0 \end{array} \right.$$



Устойчивость сети Хопфилда

Сеть с обратными связями является устойчивой, если ее матрица симметрична и имеет нули на главной диагонали, т. е. если $w_{ij} = w_{ji}$ и $w_{ii} = 0$ для всех i .

Обучение сети Хопфилда по Хеббу

Один вектор

$$w_{ij} = \frac{1}{N} x_i x_j, \quad x_i = \{0, 1\}$$

M векторов

$$w_{ij} = \frac{1}{N} \sum_{k=1}^M x_i(k) x_j(k), \quad x_i(k) = \{0, 1\}$$

Метод Хопфилда

Емкость сети :

Число эталонов = 13,8% от N
нейронов при отклонении 1%
входов

Сеть Хебба хорошо запоминает
только взаимно ортогональные
примеры или близкие к ним

Обучение сети Хопфилда по методу проекций

$$WD = D$$

$$W = DD^+$$

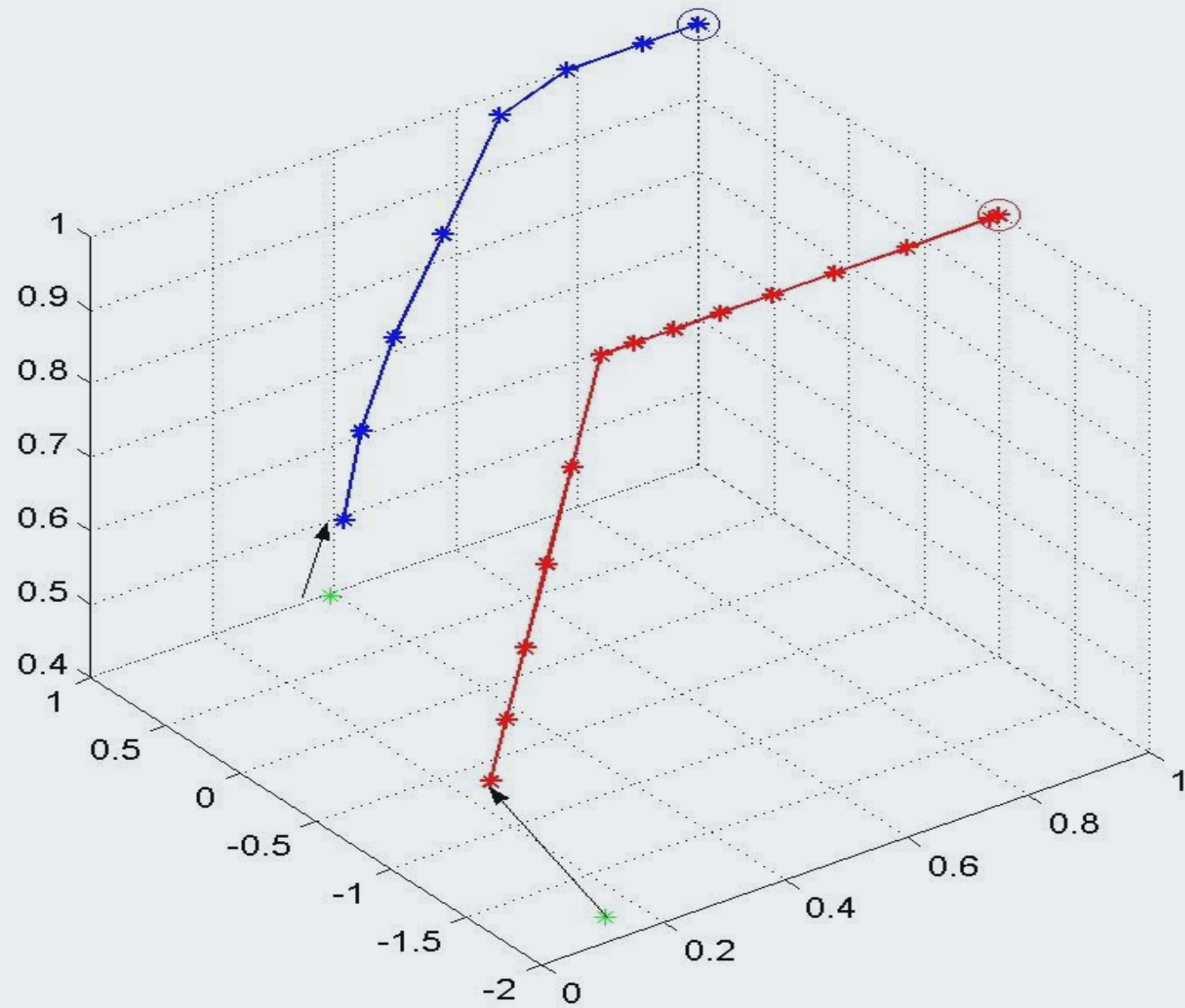
$$D^+ = (D^T D)^{-1} D^T$$

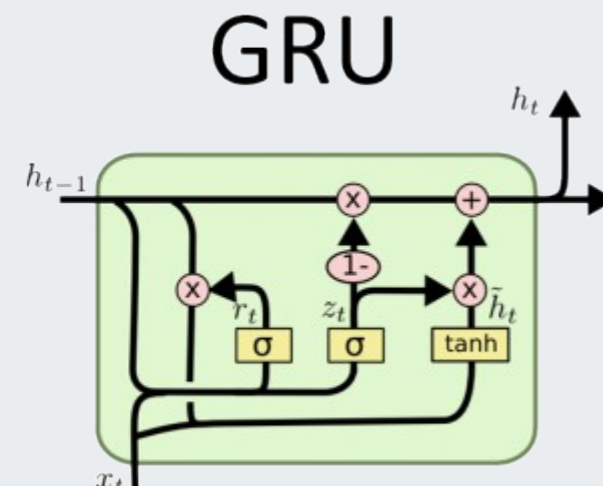
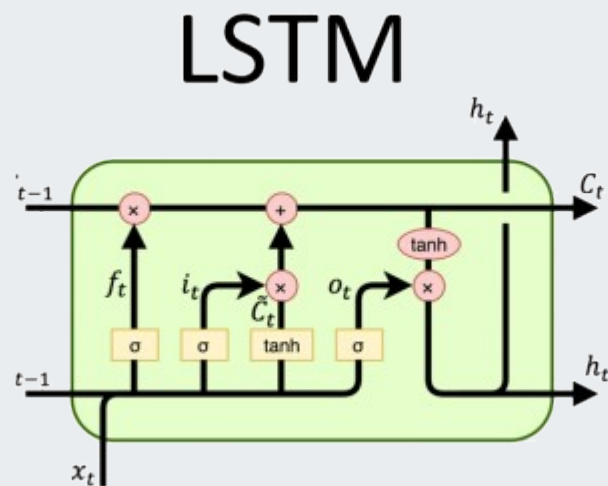
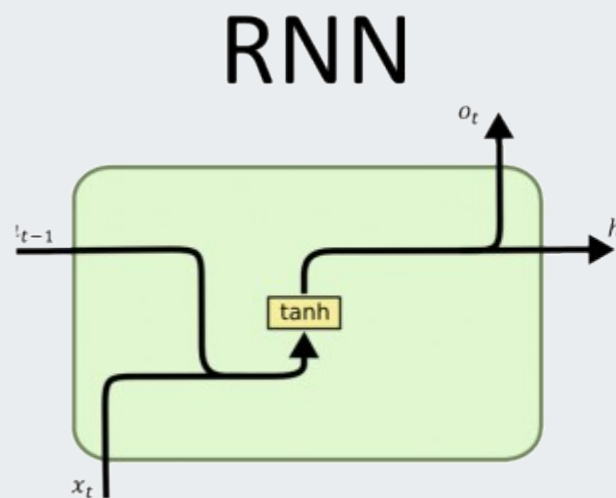
$$W = DD^T$$

$$w_{ij} = \sum_{d=1 \dots m} (y_{i,d} y_{j,d}^T)$$

увеличивает максимальную емкость сети Хопфилда до $N-1$

Сеть во времени



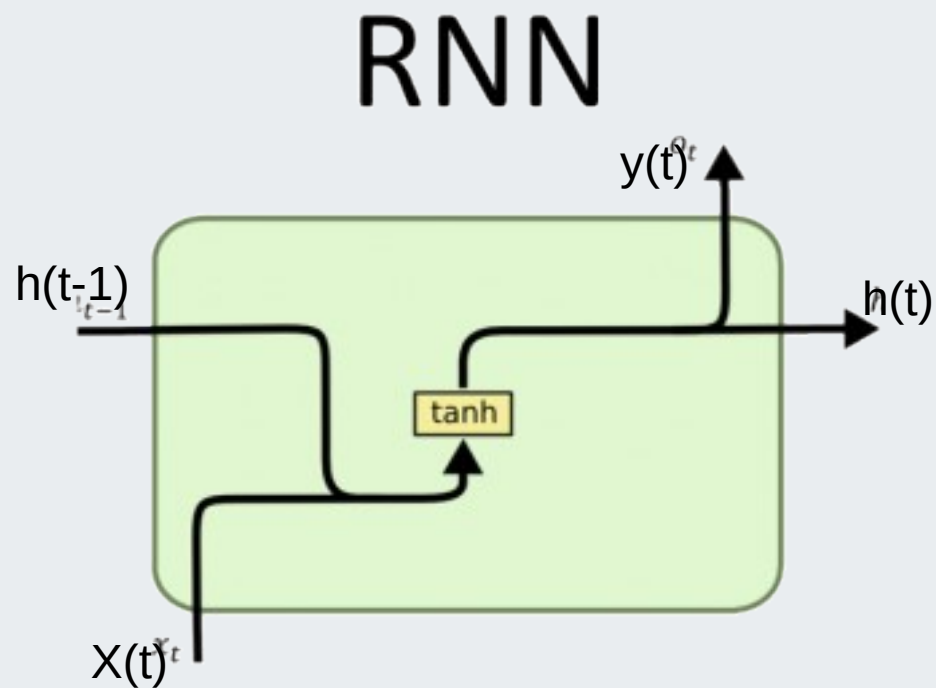


Виды рекуррентных узлов



Backpropagation Through Time

Обратное Распространение Во Времени



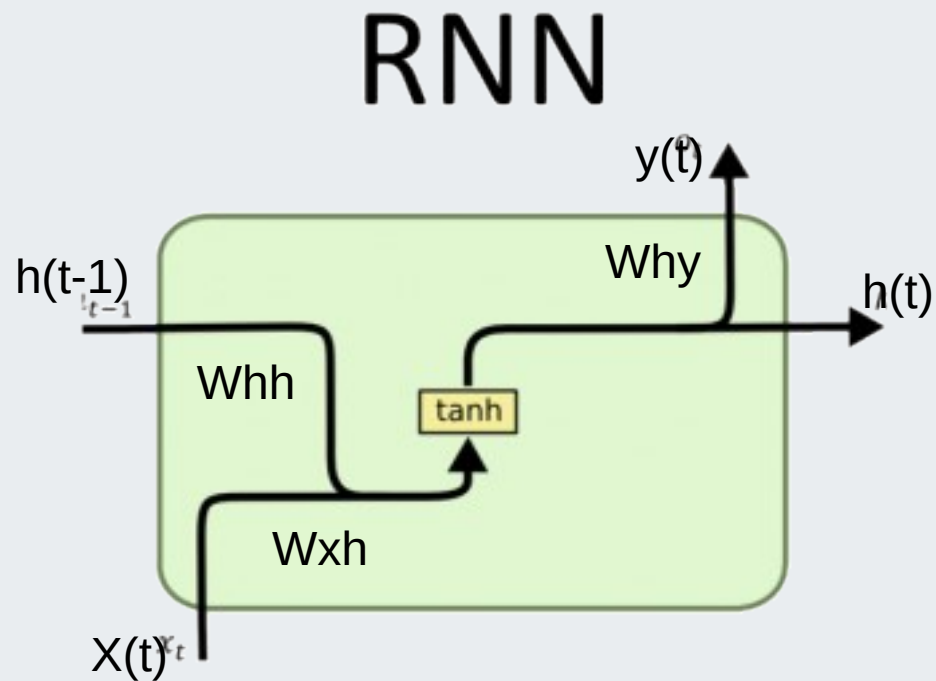
$$h(t) = \tanh(W_{xh} \cdot X(t) + W_{hh} \cdot h(t-1) + b_h)$$

$$y(t) = W_{hy} \cdot h(t) + b_y$$

$$P_c = \text{softmax}(y(t))$$

$$L = -\log(P_c)$$

Обратное Распространение Во Времени



Инициализация весовых матриц :

- инициализации W_{xh} (вход-скрытый),
- W_{hh} (скрытый-скрытый),
- W_{hy} (скрытый-выход).

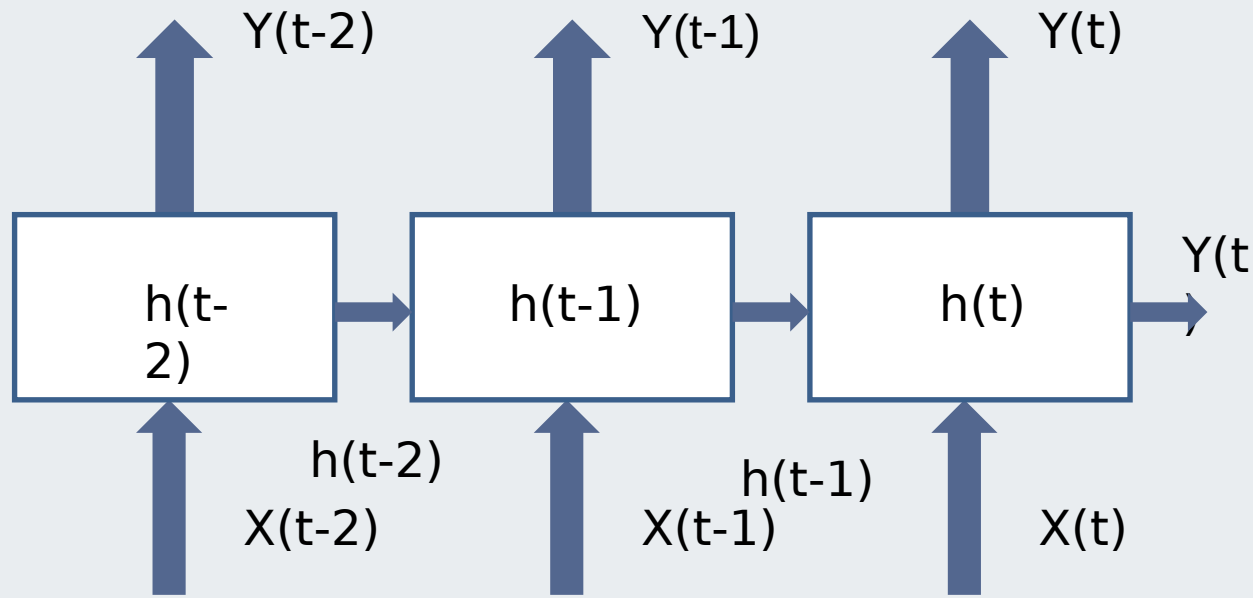
прямой проход :

$y(n), h(n), (x(t), h(t), y(t), t = 1, n)$

обратный проход ищем :

$$\frac{\partial L(n)}{\partial W_{xh}}, \frac{\partial L(n)}{\partial W_{hh}}, \frac{\partial L(n)}{\partial W_{hy}}, \frac{\partial L(n)}{\partial b_h}, \frac{\partial L(n)}{\partial b_y}$$

Обратное Распространение Во Времени



прямой проход :

$y(n), h(n), (x(t), h(t), y(t), t = 1, n)$

обратный проход ищем :

$$\frac{\partial L(n)}{\partial W_{xh}}, \frac{\partial L(n)}{\partial W_{hh}}, \frac{\partial L(n)}{\partial W_{hy}}, \frac{\partial L(n)}{\partial b_h}, \frac{\partial L(n)}{\partial b_y}$$

$$\frac{\partial L(n)}{\partial W_{hy}} = \frac{\partial L(n)}{\partial y(n)} \frac{\partial y(n)}{\partial W_{hy}}, \quad y(n) = W_{hy} h(n) + b_y$$

$$\frac{\partial y(n)}{\partial W_{hy}} = h(n), \quad \frac{\partial y(n)}{\partial b_y} = 1$$

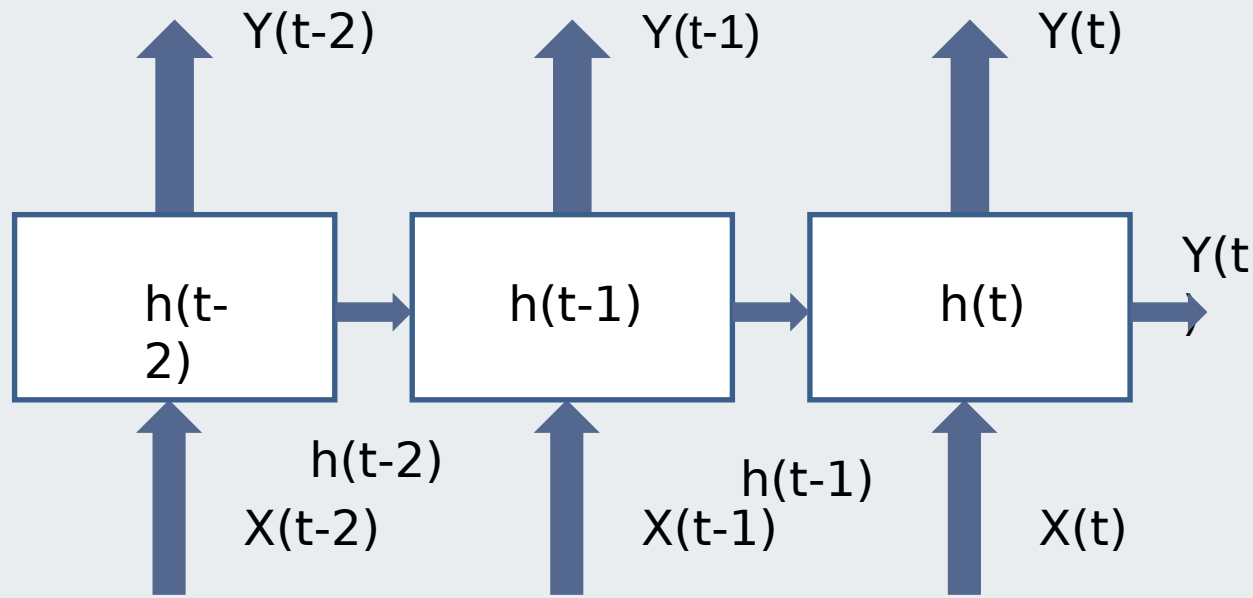
$$\frac{\partial L(n)}{\partial W_{hy}} = \frac{\partial L(n)}{\partial y(n)} h(n), \quad \frac{\partial L(n)}{\partial b_y} = \frac{\partial L(n)}{\partial y(n)}$$

$$h(t) = \tanh(W_{xh} \cdot X(t) + W_{hh} \cdot h(t-1) + b_h)$$

$$y(t) = W_{hy} \cdot h(t) + b_y$$

Backpropagation Through Time (BPTT):

Обратное Распространение Во Времени



прямой проход:

$y(n), h(n), (x(t), h(t), y(t), t = 1, n)$

обратный проход ищем:

$$\frac{\partial L(n)}{\partial W_{xh}}, \frac{\partial L(n)}{\partial W_{hh}}, \frac{\partial L(n)}{\partial W_{hy}}, \frac{\partial L(n)}{\partial b_h}, \frac{\partial L(n)}{\partial b_y}$$

$$\frac{\partial L(n)}{\partial W_{hy}} = \frac{\partial L(n)}{\partial y(n)} \frac{\partial y(n)}{\partial W_{hy}}, \quad y(n) = W_{hy}h(n) + b_y$$

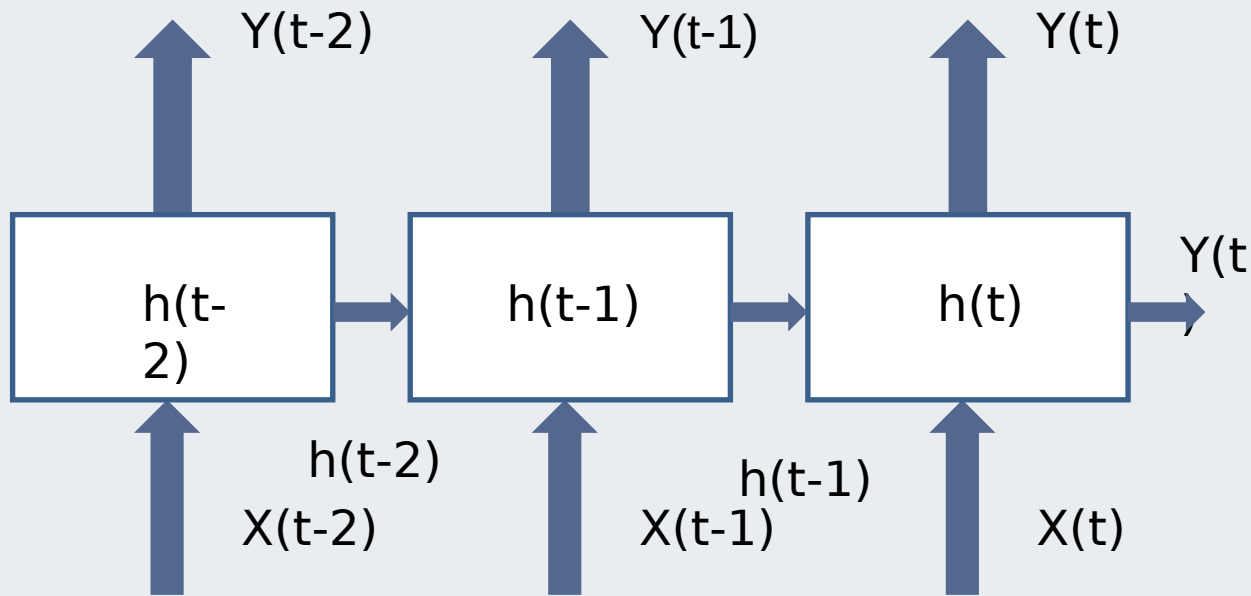
$$\frac{\partial y(n)}{\partial W_{hy}} = h(n), \quad \frac{\partial y(n)}{\partial b_y} = 1$$

$$\frac{\partial L(n)}{\partial W_{hy}} = \frac{\partial L(n)}{\partial y(n)} h(n),$$

$$\frac{\partial L(n)}{\partial b_y} = \frac{\partial L(n)}{\partial y(n)}$$

Backpropagation Through Time (BPTT):

Обратное Распространение Во Времени



прямой проход:

$y(n), h(h), (x(t), h(t), y(t), t = 1, n)$

обратный проход ищем:

$$\frac{\partial L(n)}{\partial W_{xh}}, \frac{\partial L(n)}{\partial W_{hh}}, \frac{\partial L(n)}{\partial W_{hy}}, \frac{\partial L(n)}{\partial b_h}, \frac{\partial L(n)}{\partial b_y}$$

$$\frac{\partial L(n)}{\partial W_{xh}} = \frac{\partial L(n)}{\partial y(n)} \sum_t \frac{\partial y(t)}{\partial h(t)} \frac{\partial h(t)}{\partial W_{xh}}$$

$$\frac{\partial h(t)}{\partial W_{xh}} = (1 - h(t)^2) X(t)$$

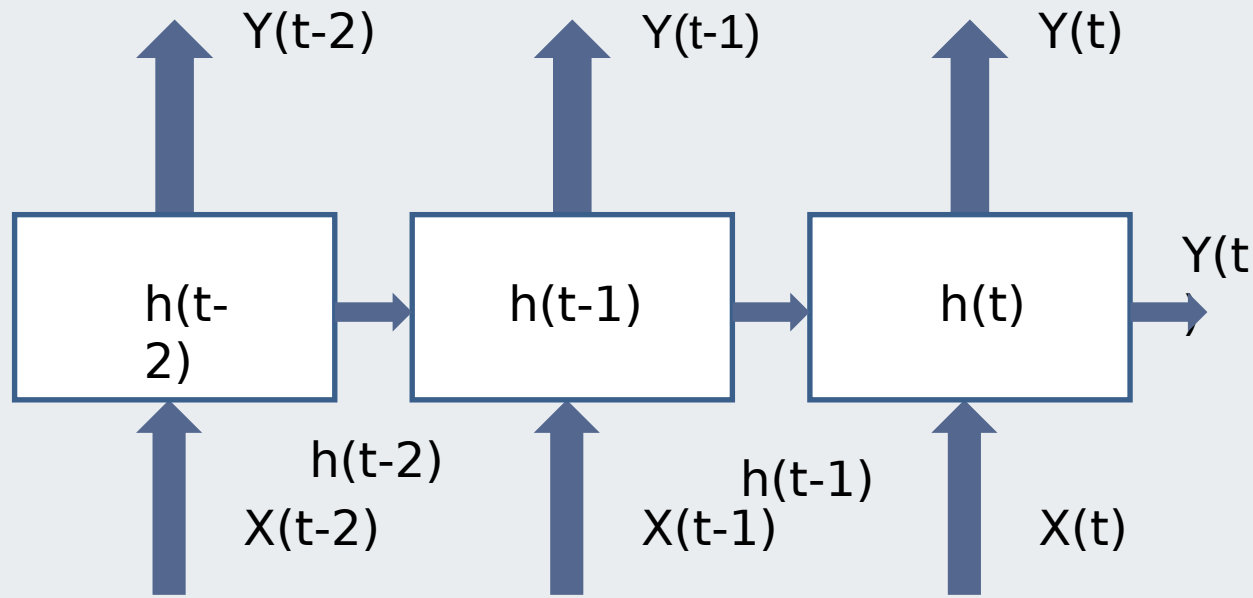
$$\frac{\partial L(n)}{\partial W_{xh}} = \frac{\partial L(n)}{\partial y(n)} \sum_t \frac{\partial y(t)}{\partial h(t)} (1 - h(t)^2) X(t)$$

$$h(t) = \tanh(W_{xh} \cdot X(t) + W_{hh} \cdot h(t-1) + b_h)$$

$$y(t) = W_{hy} \cdot h(t) + b_y$$

Backpropagation Through Time (BPTT):

Обратное Распространение Во Времени



прямой проход :

$y(n), h(n), (x(t), h(t), y(t), t = 1, n)$

обратный проход ищем :

$$\frac{\partial L(n)}{\partial W_{xh}}, \frac{\partial L(n)}{\partial W_{hh}}, \frac{\partial L(n)}{\partial W_{hy}}, \frac{\partial L(n)}{\partial b_h}, \frac{\partial L(n)}{\partial b_y}$$

$$\frac{\partial L(n)}{\partial W_{hh}} = \frac{\partial L(n)}{\partial y(n)} \sum_t \frac{\partial y(t)}{\partial h(t)} \frac{\partial h(t)}{\partial W_{hh}}$$

$$\frac{\partial h(t)}{\partial W_{hh}} = (1 - h(t)^2) h(t - 1)$$

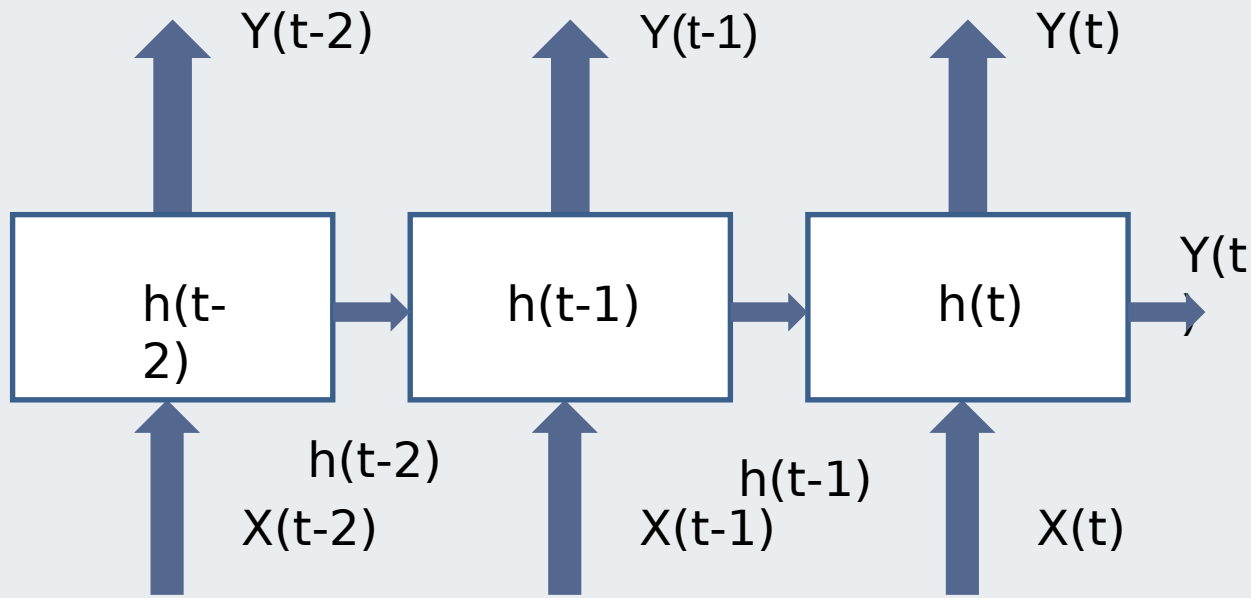
$$\frac{\partial L(n)}{\partial W_{hh}} = \frac{\partial L(n)}{\partial y(n)} \sum_t \frac{\partial y(t)}{\partial h(t)} (1 - h(t)^2) h(t - 1)$$

$$h(t) = \tanh(W_{xh} \cdot X(t) + W_{hh} \cdot h(t - 1) + b_h)$$

$$y(t) = W_{hy} \cdot h(t) + b_y$$

Backpropagation Through Time (BPTT):

Обратное Распространение Во Времени



прямой проход:

$y(n), h(h), (x(t), h(t), y(t), t = 1, n)$

обратный проход ищем:

$$\frac{\partial L(n)}{\partial W_{xh}}, \frac{\partial L(n)}{\partial W_{hh}}, \frac{\partial L(n)}{\partial W_{hy}}, \frac{\partial L(n)}{\partial b_h}, \frac{\partial L(n)}{\partial b_y}$$

$$\frac{\partial L(n)}{\partial b_h} = \frac{\partial L(n)}{\partial y(n)} \sum_t \frac{\partial y(t)}{\partial h(t)} \frac{\partial h(t)}{\partial b_h}$$

$$\frac{\partial h(t)}{\partial b_h} = (1 - h(t)^2)$$

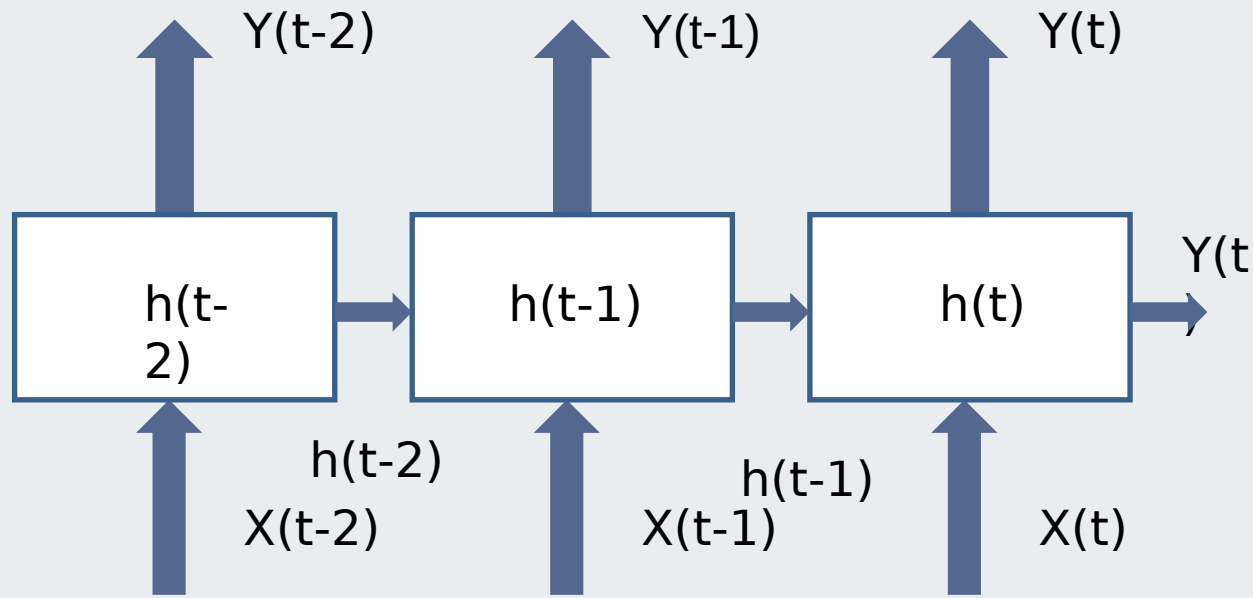
$$\frac{\partial L(n)}{\partial b_h} = \frac{\partial L(n)}{\partial y(n)} \sum_t \frac{\partial y(t)}{\partial h(t)} (1 - h(t)^2)$$

$$h(t) = \tanh(W_{xh} \cdot X(t) + W_{hh} \cdot h(t-1) + b_h)$$

$$y(t) = W_{hy} \cdot h(t) + b_y$$

Backpropagation Through Time (BPTT):

Обратное Распространение Во Времени



прямой проход:

$y(n), h(n), (x(t), h(t), y(t), t = 1, n)$

обратный проход ищем:

$$\frac{\partial L(n)}{\partial W_{xh}}, \frac{\partial L(n)}{\partial W_{hh}}, \frac{\partial L(n)}{\partial W_{hy}}, \frac{\partial L(n)}{\partial b_h}, \frac{\partial L(n)}{\partial b_y}$$

$$\frac{\partial y(t)}{\partial h(t)} = \frac{\partial y(t)}{\partial h(t+1)} \frac{\partial h(t+1)}{\partial h(t)} = \frac{\partial y(t)}{\partial h(t+1)} (1 - h(t)^2) W_{hh}$$

$$\frac{\partial y(n)}{\partial h(n)} = W_{hy}$$

$$\frac{\partial L(n)}{\partial y(n)}, \quad L(n) = -\ln(P_c), \quad P_c = \text{softmax}(y(n))$$

$$\frac{\partial L(n)}{\partial y(n)} = \begin{cases} P_c, y_{\text{true}} = C \\ P_c - 1, y_{\text{true}} \neq C \end{cases}$$

Обратное Распространение Во Времени

Повторяем с каждым примером

1. Установите входные данные
2. Установите начальное скрытое состояние (вектор нулей).
3. Сделайте прямой проход
4. Оцените потери
5. Определите производную потерь на выходе
6. Продвигайте ошибку вглубь сети
7. Накапливайте поправку для параметров сети
8. Пришли в первый узел -> делаем поправку параметров сети

Обратное Распространение Во Времени

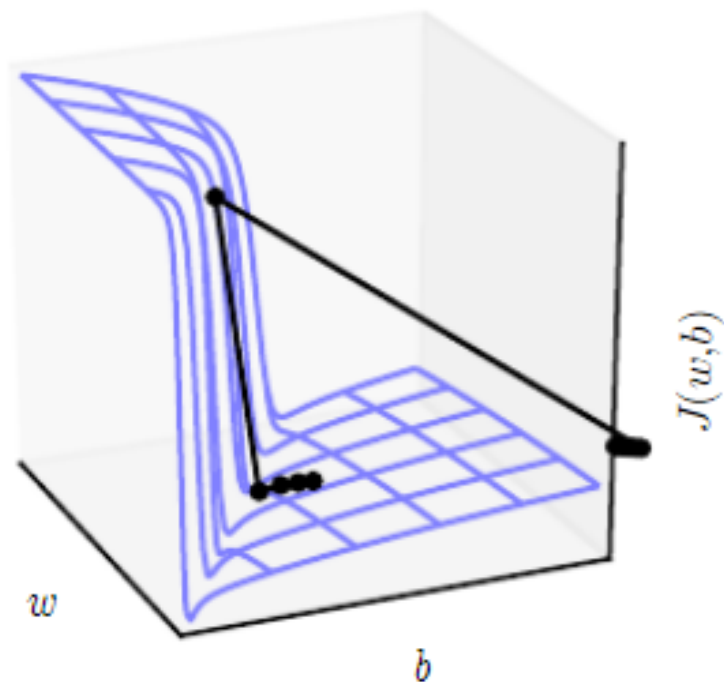
Упрощения:

1. Объединяем $\frac{\partial L}{\partial y} \frac{\partial y}{\partial h} = \frac{\partial L}{\partial h}$
2. Закончив с обратным распространением во времени ВРТТ, используем обрезку на значениях градиента ниже - 1 или выше 1. Это поможет избавиться от проблемы со **взрывными градиентами**.
3. Когда все градиенты подсчитаны, обновляем параметры веса и смещения, используя градиентный спуск

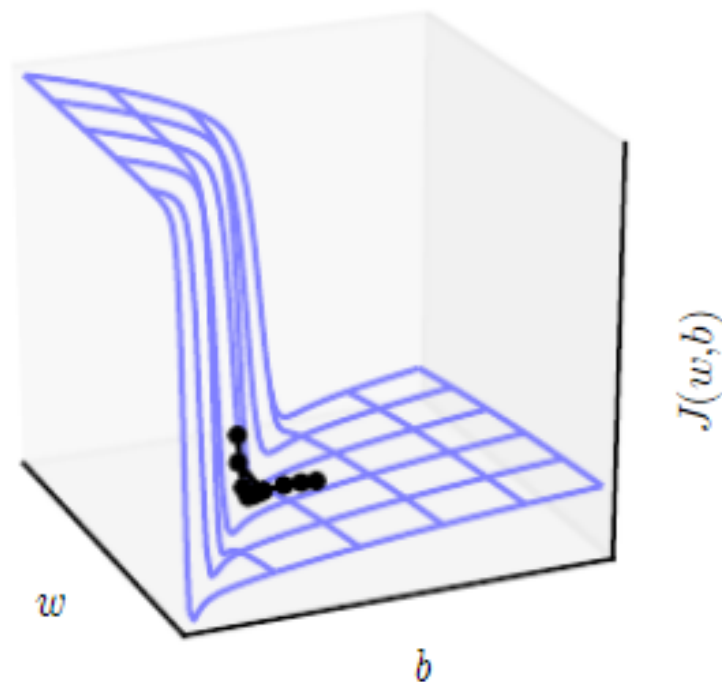
Градиентный взрыв

<https://www.deeplearningbook.org/contents/rnn.html>

Without clipping



With clipping

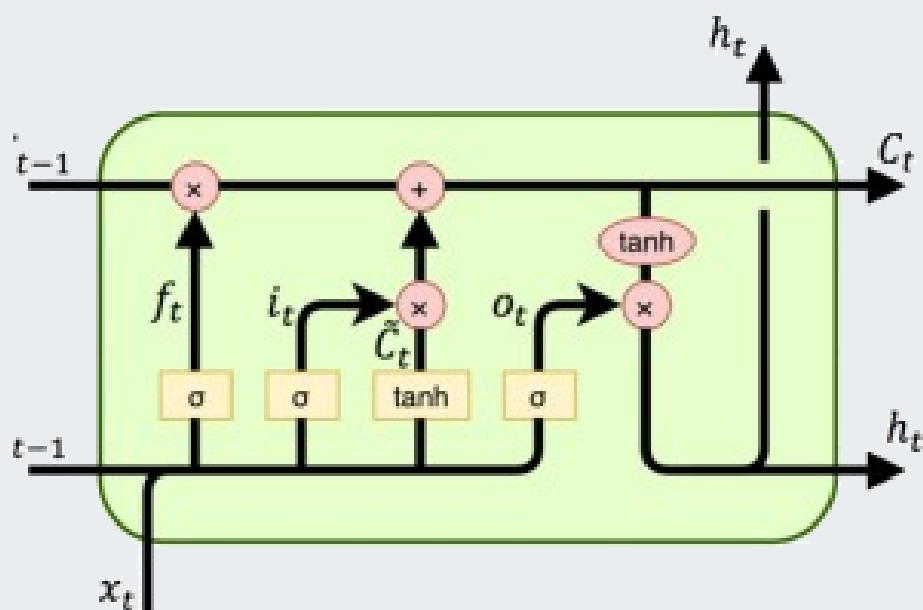




Long Short-Term Memory

Модель узла

LSTM



- σ_g : на основе **сигмоиды**.
- σ_c : на основе **гиперболического тангенса**.
- σ_h : на основе гиперболического тангенса.

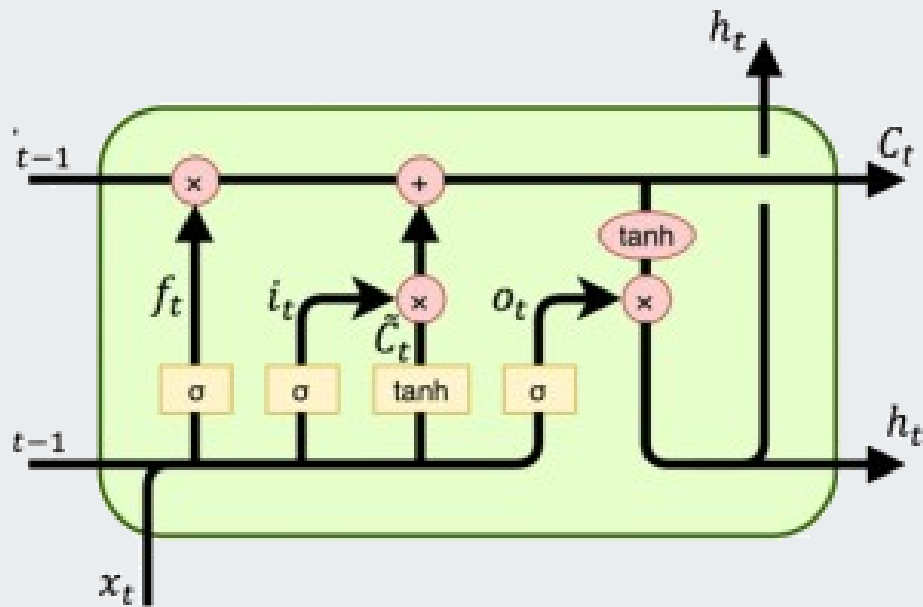
$$\begin{aligned}f_t &= \sigma_g(W_f x_t + U_f h_{t-1} + b_f) \\i_t &= \sigma_g(W_i x_t + U_i h_{t-1} + b_i) \\o_t &= \sigma_g(W_o x_t + U_o h_{t-1} + b_o) \\c_t &= f_t \circ c_{t-1} + i_t \circ \sigma_c(W_c x_t + U_c h_{t-1} + b_c) \\h_t &= o_t \circ \sigma_h(c_t)\end{aligned}$$

Переменные:

- x_t — входной вектор,
- h_t — выходной вектор,
- c_t — вектор состояний,
- W, U и b — матрицы параметров и вектор,
- f_t, i_t и o_t — векторы вентилей,
 - f_t — вектор вентиля забывания, вес запоминания старой информации,
 - i_t — вектор входного вентиля, вес получения новой информации,
 - o_t — вектор выходного вентиля, кандидат на выход.

BPTT: LSTM

LSTM



<https://nicodjimenez.github.io/2014/08/08/lstm.html>

$$X_{ct} = [X_t, h_{t-1}]$$

$$C_t = \phi(W_c X_{ct} + b_c)$$

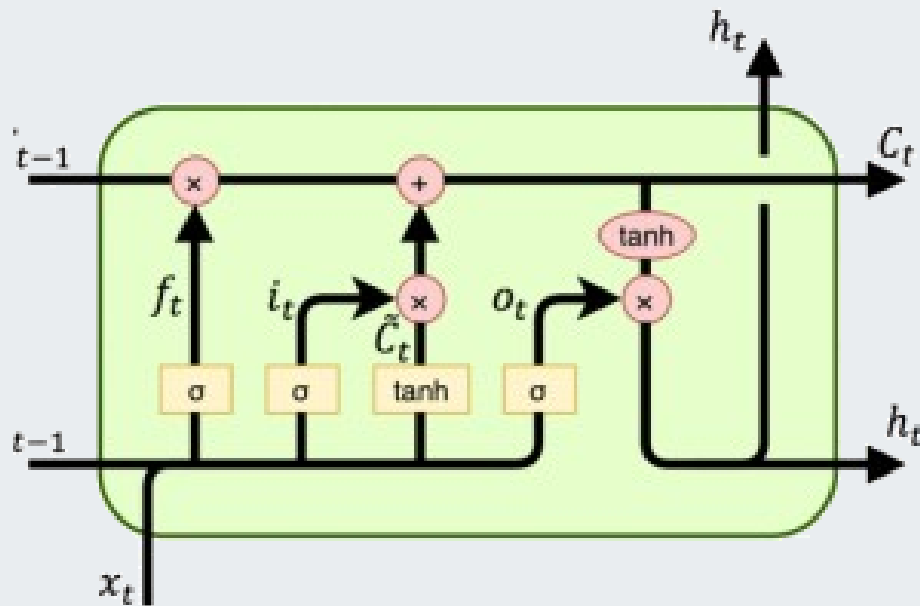
$$i_t = \sigma(W_i X_{ct} + b_i)$$

$$f_t = \sigma(W_f X_{ct} + b_f)$$

$$o_t = \sigma(W_o X_{ct} + b_o)$$

BPTT: LSTM

LSTM



<https://nicodjimenez.github.io/2014/08/08/lstm.html>

$$L_t = \|h_t - y_t\|^2, \quad L = \sum_t L_t$$

$$\frac{\partial L}{\partial W} = \sum_t^T \sum_i^M \frac{\partial L}{\partial h_{it}} \frac{\partial h_{it}}{\partial W},$$

M - число ячеек

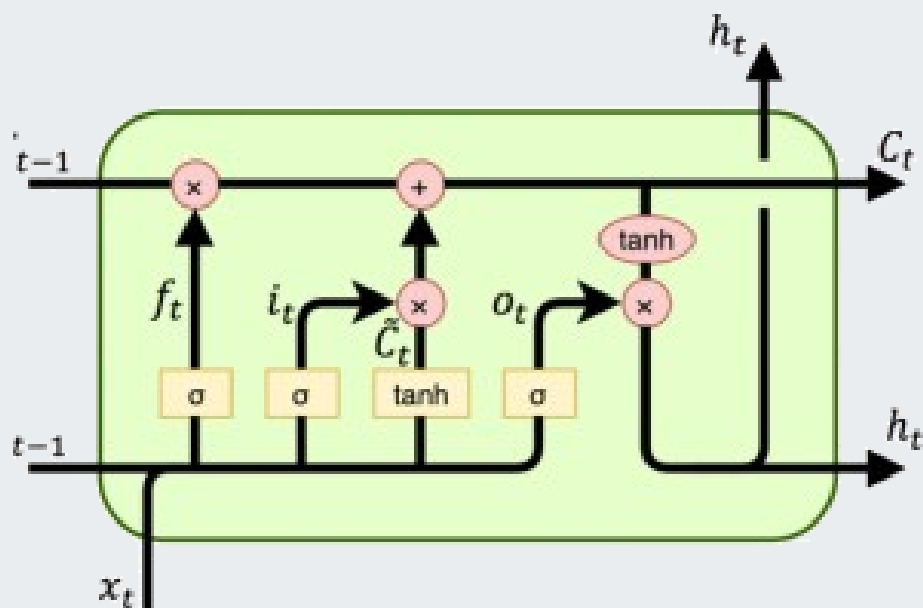
T - длина последовательности

$$\frac{\partial L}{\partial h_{it}} = \sum_{s=t,T} \frac{\partial L_s}{\partial h_{it}}, \quad L_t = \begin{cases} L_t + L_{t+1}, & t < T \\ L_t, & t = T \end{cases}$$

$$\frac{\partial L}{\partial h_t} = \frac{\partial L_t}{\partial h_t} + \frac{\partial L_{t+1}}{\partial h_t}, \quad \frac{\partial L}{\partial h_T} = \frac{\partial L_T}{\partial h_T}$$

Модель узла

LSTM



«карусель константной ошибки»
(constant error carousel)

$$f_t = 1$$

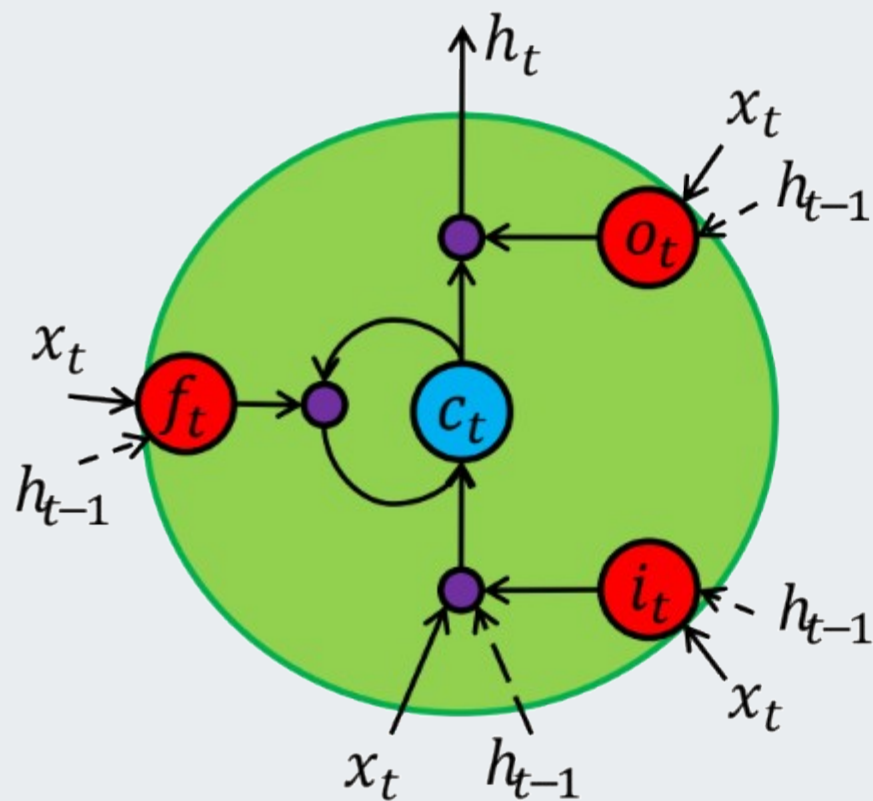
$$C_t = f_t \circ C_{t-1} + i_t \circ \tanh(W_{xc}X_t + W_{hc}h_t - 1 + b_c)$$

$$C_t = C_{t-1} + i_t \circ \tanh(W_{xc}X_t + W_{hc}h_t - 1 + b_c)$$

$$\frac{\partial C_t}{\partial C_{t-1}} = 1$$

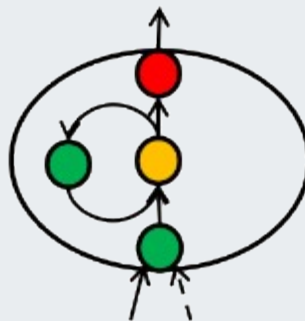
$$b_f > 1$$

Модель узла: гейты

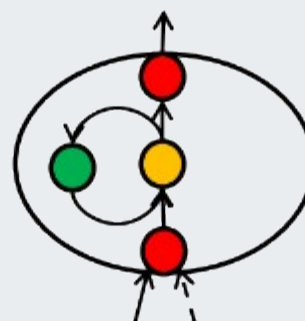


- - gate is close
- - gate is open

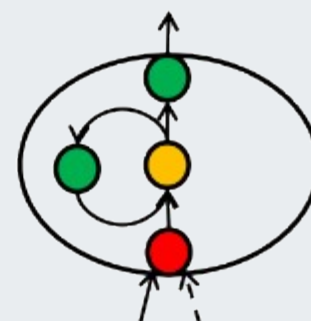
Captures info



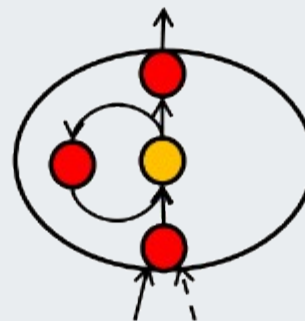
Keeps info



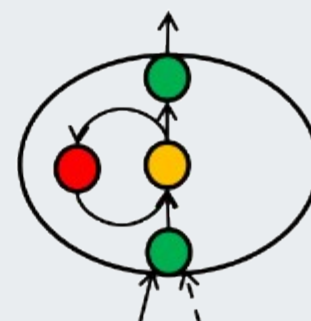
Releases info



Erases info



= RNN



LSTM: Особенности

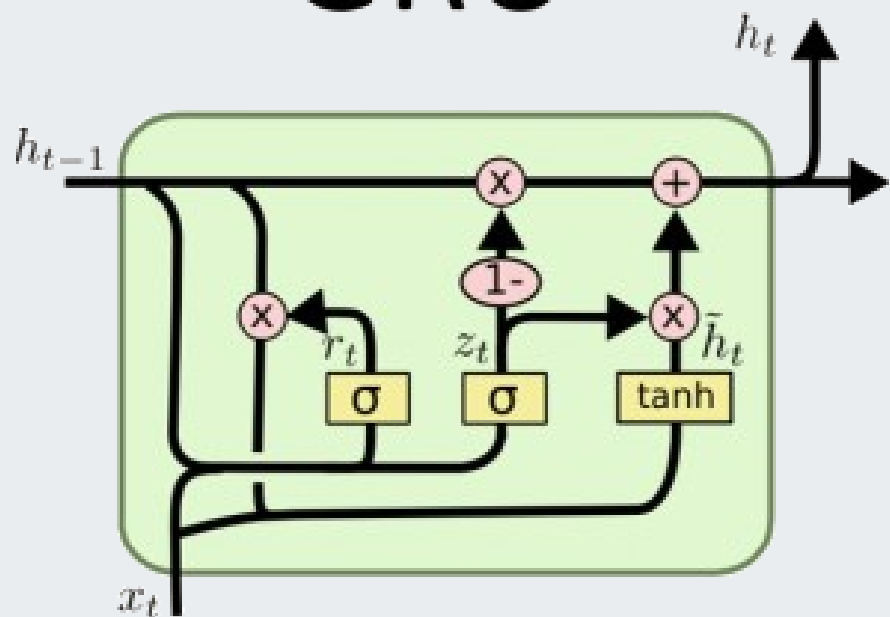
1. Много вариантов организации:
 1. «Замочные скважины» (11 матриц параметров)
 2. Дополнительные связи
 3. Без некоторых гейтов
 4. Двухнаправленные
2. Некоторые простые архитектуры (без одного из гейтов!) не хуже «ванильного» LSTM.
3. Градиент взрывается (но хотя бы не затухает) – делаем обрезку
4. Инициализация не всегда простая

Gated Recurrent Unit



Модель узла

GRU



◦ обозначает произведение Адамара. $h_0 = 0$.

$$z_t = \sigma_g(W_z x_t + U_z h_{t-1} + b_z)$$

$$r_t = \sigma_g(W_r x_t + U_r h_{t-1} + b_r)$$

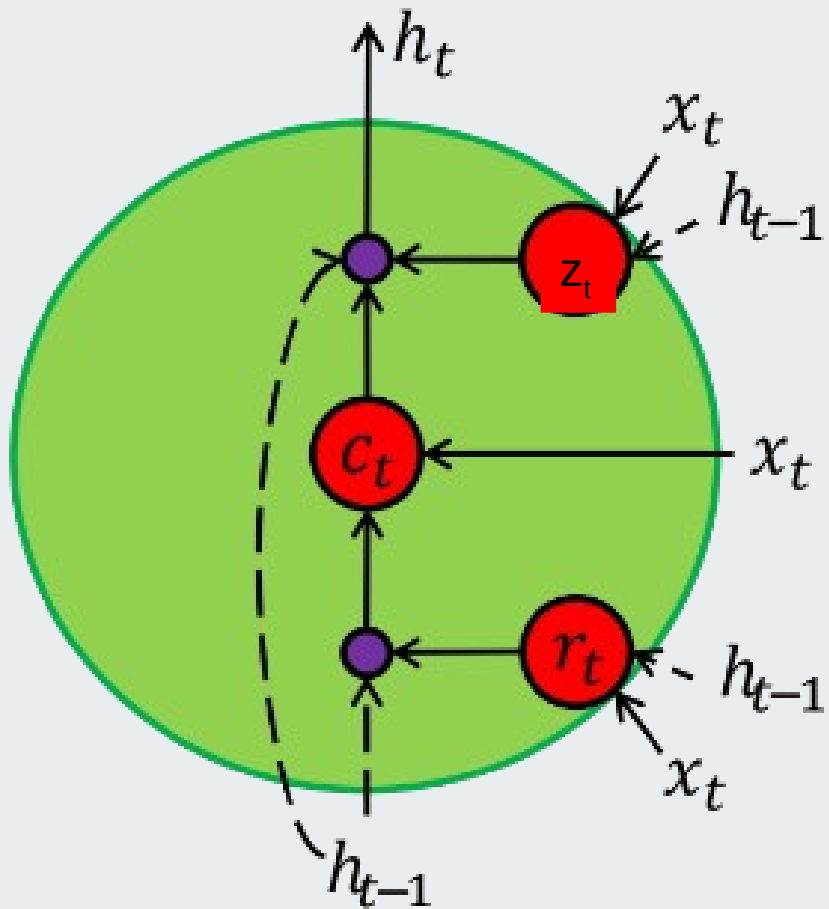
$$h_t = z_t \circ h_{t-1} + (1 - z_t) \circ \sigma_h(W_h x_t + U_h(r_t \circ h_{t-1}) + b_h)$$

Переменные

- x_t : входной вектор
- h_t : выходной вектор
- z_t : вектор вентиля обновления
- r_t : вектор вентиля сброса
- W , U и b : матрицы параметров и вектор

A 5x10 grid of dots representing a coordinate plane. The dots are arranged in 5 horizontal rows and 10 vertical columns, forming a rectangular grid.

Протекание градиента: Карусель



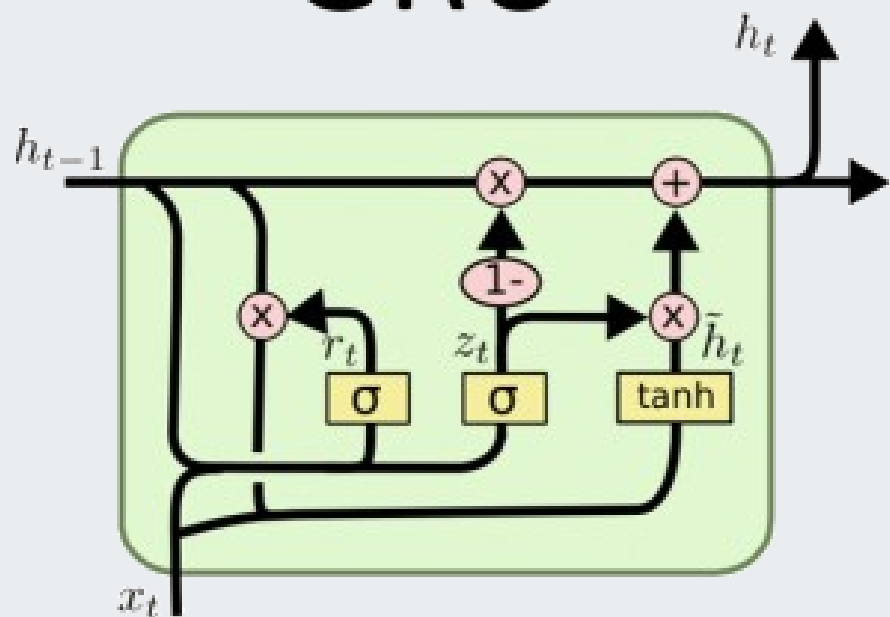
$$c_t = \sigma(W_h \mathbf{x}_t + U_h (\mathbf{h}_{t-1} \circ \mathbf{r}_t) + \mathbf{b}_h)$$

$$h_t = (1 - z_t) \circ c_t + z_t \circ h_{t-1}$$

$$\frac{\partial h_{t+1}}{\partial h_t} = z_{t+1} \circ \frac{\partial c_{t+1}}{\partial h_t} + z_{t+1}, \quad b_z > 1$$

GRU: Особенности

GRU



1. Меньше матриц (6)
2. Меньше весов
3. Чуть хуже LSTM
4. Внутренний слой для многослойной модели.

RNN: Особенности

1. RNN имеют довольно простую общую структуру: уровни LSTM или GRU.
2. Все они выдают последовательность выходов, кроме верхнего.
3. DropOut и Batchnorm между слоями, и осторожно на рекуррентных связях
4. Слоеv немного.
5. Skip-layer connections, как ResNet.
6. Состояния остаются, если нужно (в Keras : `stateful=True`)



ПРОБЛЕМЫ ОБУЧЕНИЯ РЕКУРРЕНТНЫХ СЕТЕЙ

1. Взрыв градиента
2. Выбор модели инициализации
3. Выбор шага
4. Включение регуляризации
5. Сложности с нормализацией

RNN: Проблемы

- Невозможность пересмотреть решения по хранению
- Ограниченная емкость хранилища
- Отсутствие Распараллеливаемости

xLSTM



xLSTM: Особенности

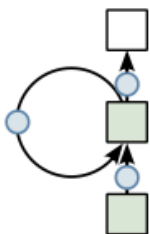
LSTM

Memory Cells

- Constant Error Carousel
- Sigmoid Gating
- Recurrent Inference
- Recurrent Training

$$c_t = f_t c_{t-1} + i_t z_t$$

$$h_t = o_t \psi(c_t)$$



Memory Cells

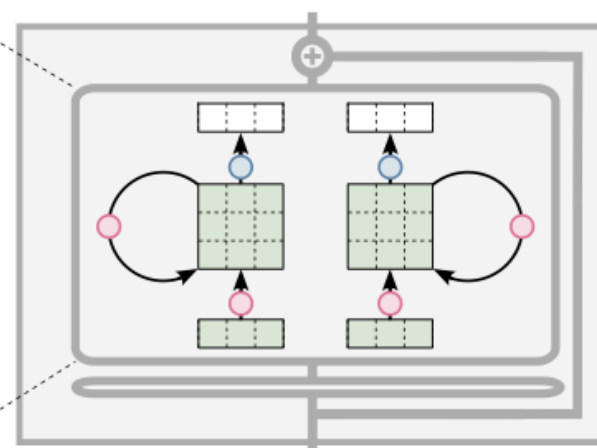
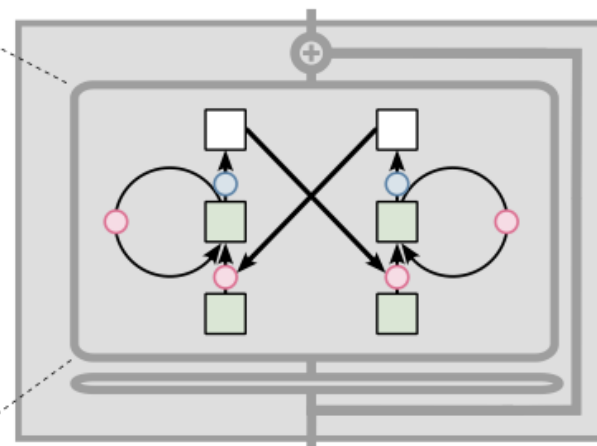
sLSTM

- + Exponential Gating
- + New Memory Mixing

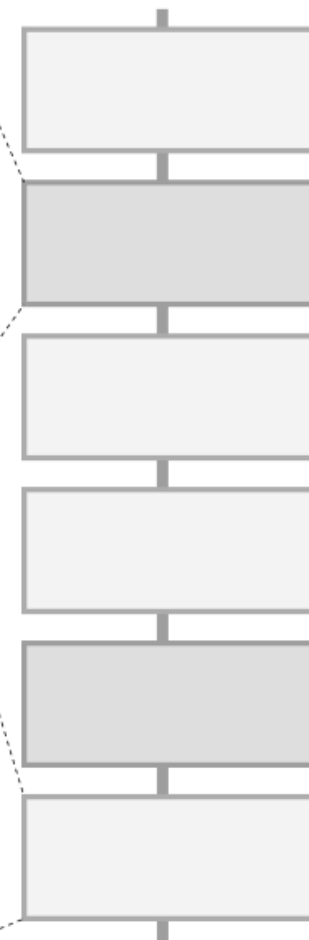
mLSTM

- + Exponential Gating
- + Matrix Memory
- + Parallel Training
- + Covariance Update Rule

xLSTM Blocks

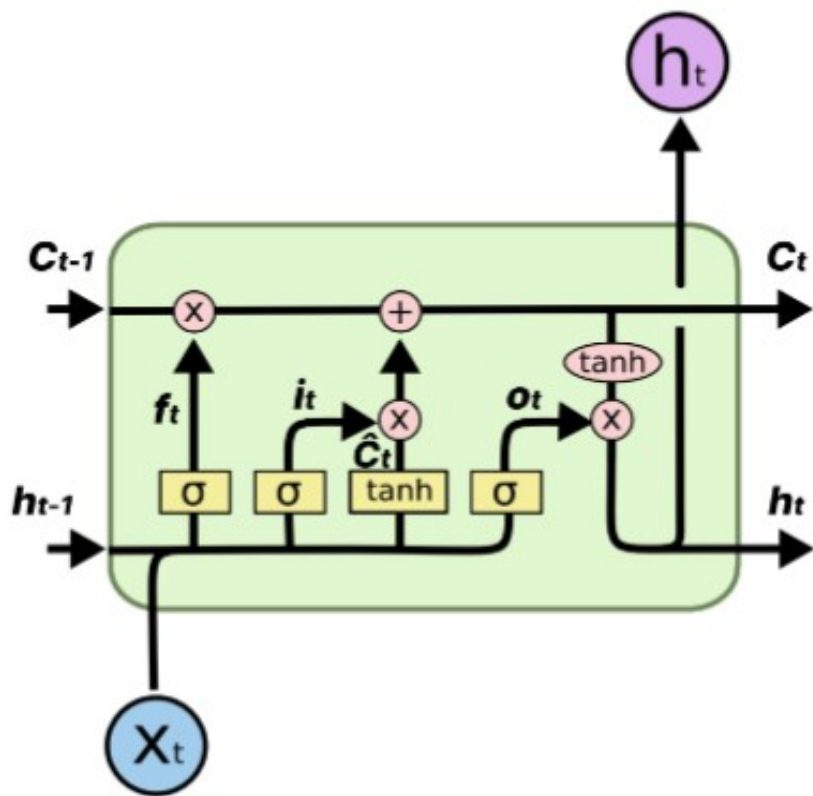


xLSTM

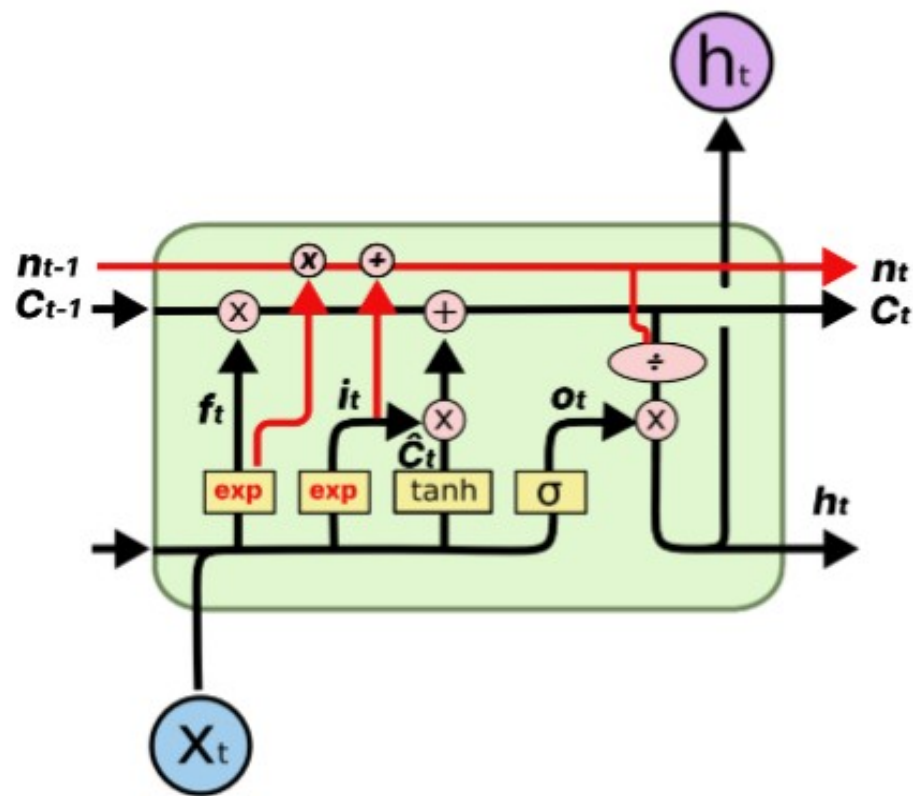


xLSTM: Особенности

Блок LSTM



Блок sLSTM



xLSTM: Особенности

LSTM

$$i_t = \sigma(\tilde{i}_t),$$

$$f_t = \sigma(\tilde{f}_t),$$

$$\tilde{i}_t = \mathbf{w}_i^\top \mathbf{x}_t + r_i h_{t-1} + b_i \quad \text{input gate}$$

$$\tilde{f}_t = \mathbf{w}_f^\top \mathbf{x}_t + r_f h_{t-1} + b_f \quad \text{forget gate}$$

sLSTM

$$m_t = \max(\log(f_t) + m_{t-1}, \log(i_t)) \quad \text{stabilizer state}$$

$$i'_t = \exp(\log(i_t) - m_t) = \exp(\tilde{i}_t - m_t) \quad \text{stabil. input gate}$$

$$f'_t = \exp(\log(f_t) + m_{t-1} - m_t) \quad \text{stabil. forget gate}$$

xLSTM: Особенности

LSTM

$$i_t = \sigma(\tilde{i}_t),$$

$$f_t = \sigma(\tilde{f}_t),$$

$$o_t = \sigma(\tilde{o}_t),$$

$$\tilde{i}_t = \mathbf{w}_i^\top \mathbf{x}_t + \underline{r_i h_{t-1}} + b_i \quad \text{input gate}$$

$$\tilde{f}_t = \mathbf{w}_f^\top \mathbf{x}_t + \underline{r_f h_{t-1}} + b_f \quad \text{forget gate}$$

$$\tilde{o}_t = \mathbf{w}_o^\top \mathbf{x}_t + \underline{r_o h_{t-1}} + b_o \quad \text{output gate}$$

mLSTM

$$i_t = \exp(\tilde{i}_t),$$

$$f_t = \sigma(\tilde{f}_t) \text{ OR } \exp(\tilde{f}_t), \quad \tilde{f}_t = \mathbf{w}_f^\top \mathbf{x}_t + b_f$$

$$o_t = \sigma(\tilde{o}_t),$$

$$\tilde{i}_t = \mathbf{w}_i^\top \mathbf{x}_t + b_i \quad \text{input gate}$$

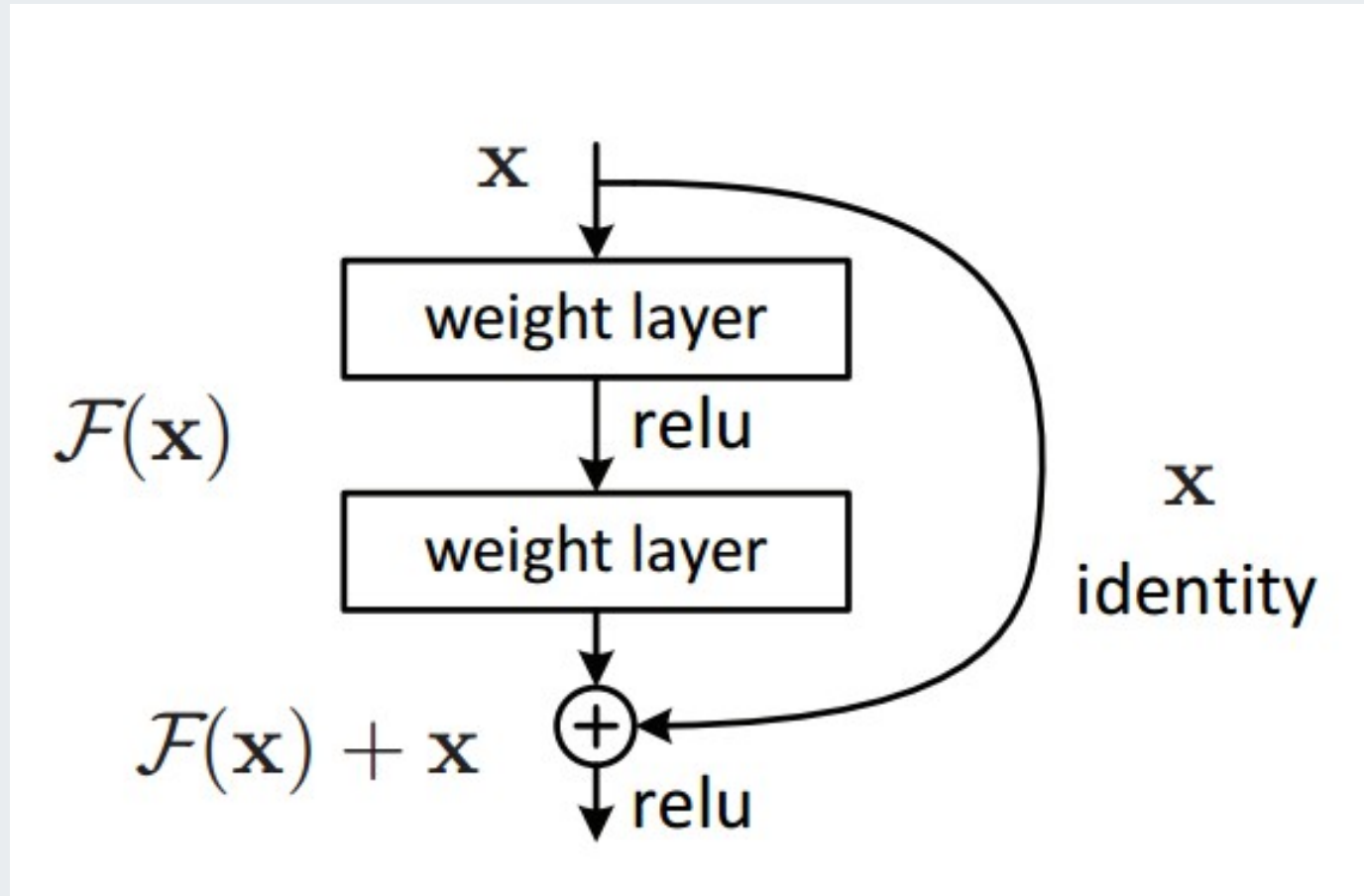
$$\tilde{f}_t = \mathbf{w}_f^\top \mathbf{x}_t + b_f \quad \text{forget gate}$$

$$\tilde{o}_t = \mathbf{W}_o \mathbf{x}_t + b_o \quad \text{output gate}$$

Матричная память состояний

xLSTM: Особенности

Резидуальный блок



xLSTM: Особенности

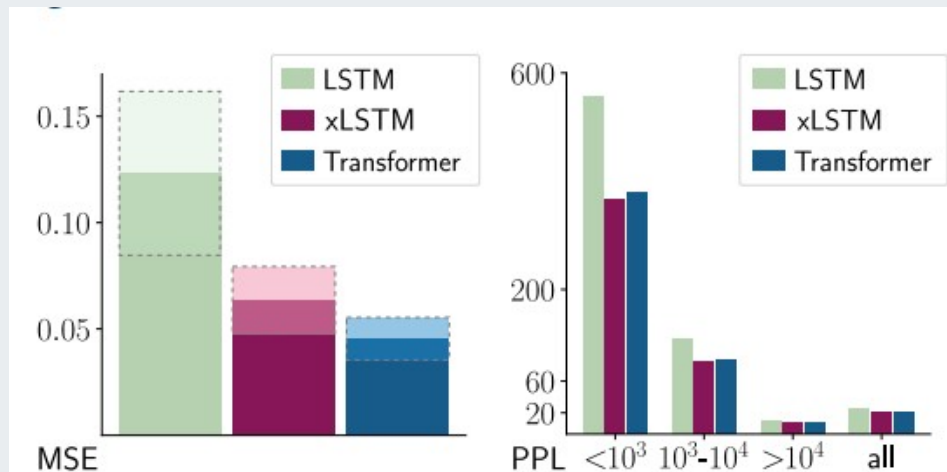
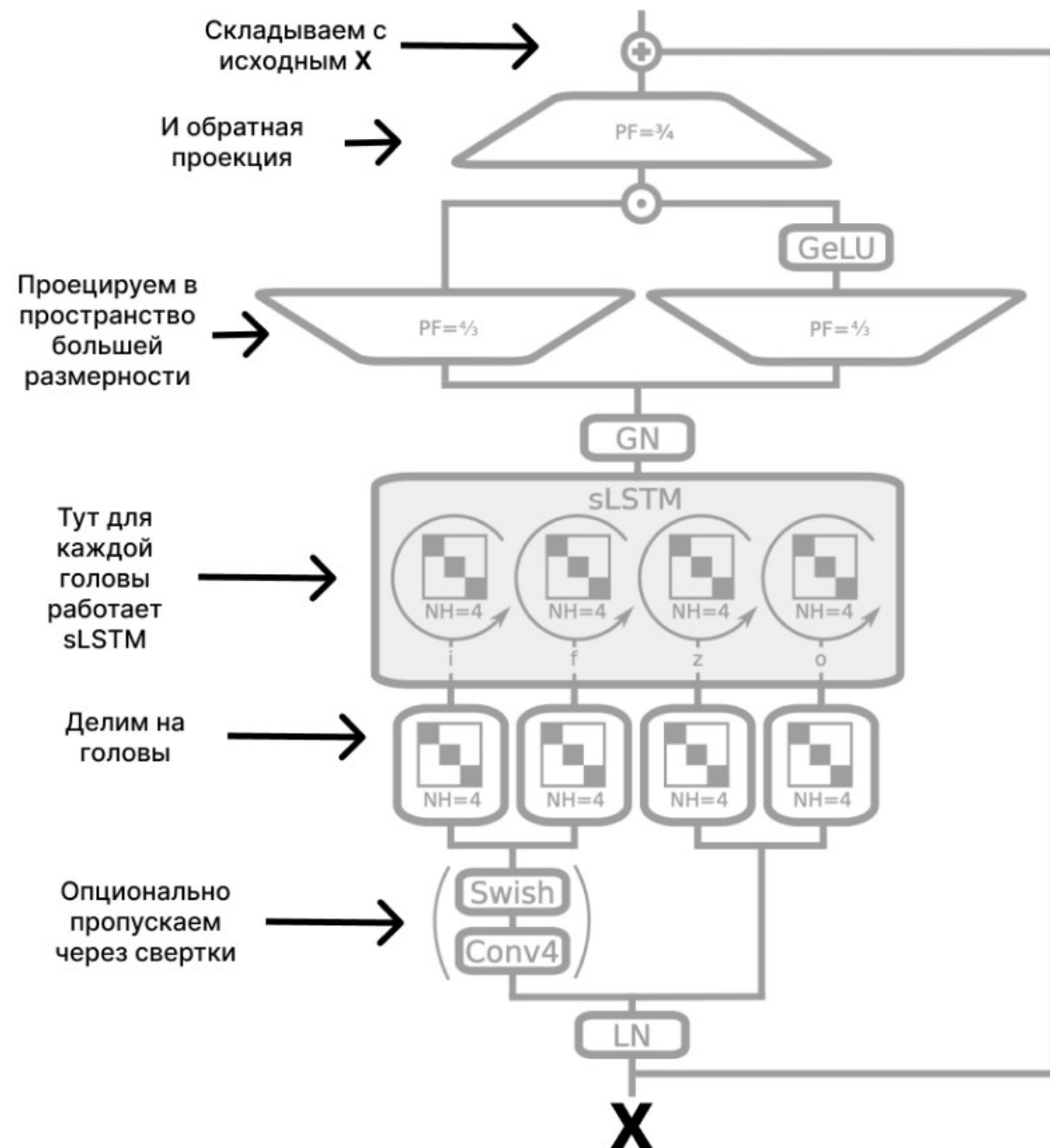


Figure 2: LSTM limitations. **Left:** Nearest Neighbor Search problem in terms of mean squared error (MSE). Given a reference vector, a sequence is scanned sequentially for the most similar vector with the objective to return its attached value at sequence end. LSTM struggles to revise a stored value when a more similar vector is found. Our new xLSTM overcomes this limitation by exponential gating. **Right:** Rare Token Prediction. The perplexity (PPL) of token prediction on Wikitext-103, in buckets of token frequency. LSTM performs worse on predicting rare tokens because of its limited storage capacities, whereas our new xLSTM solves this problem via a matrix memory.

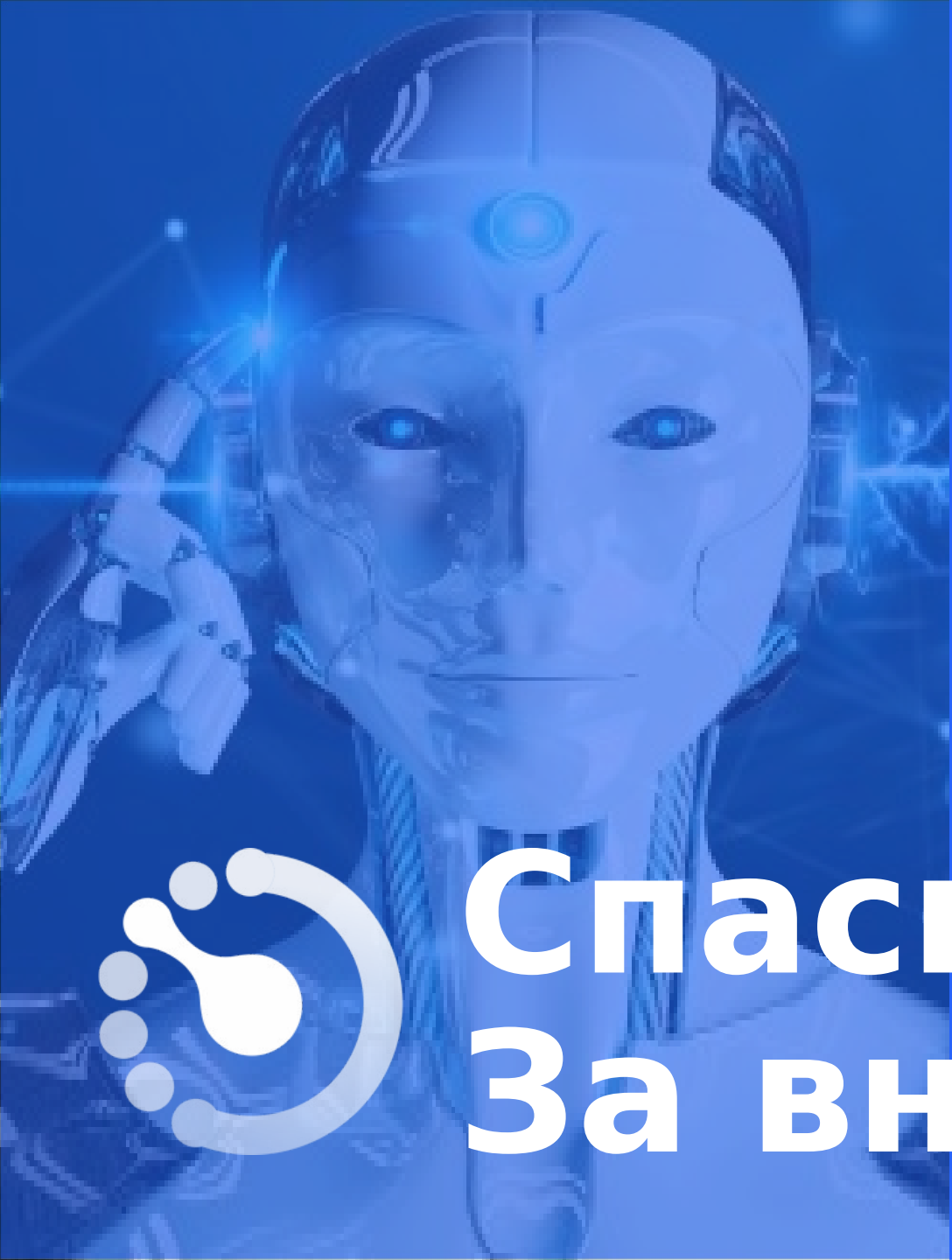


xLSTM: Особенности

Model	#Params M	SlimPajama (15B) ppl ↓
GPT-3	356	14.26
Llama	407	<u>14.25</u>
H3	420	18.23
Mamba	423	<u>13.70</u>
Hyena	435	17.59
RWKV-4	430	<u>15.62</u>
RWKV-5	456	16.53
RWKV-6	442	17.40
RetNet	431	16.23
HGRN	411	21.83
GLA	412	19.56
HGRN2	411	16.77
xLSTM[1:0]	409	<u>13.43</u>
xLSTM[7:1]	408	13.48

RNN: Приложения

1. Временные ряды
2. Языковые модели
3. Обработка видеопотока
4. Модели с вниманием



Спасибо
За внимание

